

本小节内容

尾插法新建链表实战

尾插法新建链表实战

在第 10.5 小节我们已经讲解了链表的新增，删除，查找的原理。

首先我们绘制了尾插法新建链表代码实战流程图，不会自己独立写出代码的同学，可以看看视频，看看老师写的代码，接着先画一个流程图，再根据自己画的流程图来写代码，流程图的详细情况根据个人来定制，并不一定要和老师的完全一致。



尾插法的特点是我们始终让尾指针 **r** 指向链表的尾部。

本小节是实战尾部插入来新建链表，代码保留了 11.3 节的头插法，自然是全程手撸代码，下面直接上代码：

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef int ElemType;
```

```
typedef struct LNode{
```

```
    ElemType data;
```

```
    struct LNode *next; //指向下一个结点
```

```
}LNode,*LinkList;
```

```
//头插法新建链表
```

```
LinkList list_head_insert(LinkList &L)
```



```
{

    LNode *s;int x;

    L=(LinkedList)malloc(sizeof(LNode));//带头结点的链表

    L->next=NULL;//L->data 里边没放东西

    scanf("%d",&x);//从标准输入读取数据

    //3 4 5 6 7 9999

    while(x!=9999){

        s=(LNode*)malloc(sizeof(LNode));//申请一个新空间给 s, 强制类型转换

        s->data=x;//把读取到的值, 给新空间中的 data 成员

        s->next=L->next;//让新结点的 next 指针指向链表的第一个元素 (第一个放我们数据的元素)

        L->next=s;//让 s 作为第一个元素

        scanf("%d",&x);//读取标准输入

    }

    return L;

}

//尾插法新建链表

LinkedList list_tail_insert(LinkedList &L)

{

    int x;

    L=(LinkedList)malloc(sizeof(LNode));//带头结点的链表

    LNode* s, * r = L;//LinkedList s,r=L;也可以, r 代表链表表尾结点, 指向链表尾部

    //3 4 5 6 7 9999
```



```
scanf("%d",&x);
```

```
while(x!=9999){
```

```
    s=(LNode*)malloc(sizeof(LNode));
```

```
    s->data=x;
```

```
    r->next=s;//让尾部结点指向新结点
```

```
    r=s;//r 指向新的表尾结点
```

```
    scanf("%d",&x);
```

```
}
```

```
r->next=NULL;//尾结点的 next 指针赋值为 NULL
```

```
return L;
```

```
}
```

```
//打印链表中每个结点的值
```

```
void print_list(LinkList L)
```

```
{
```

```
    L=L->next;
```

```
    while(L!=NULL)//NULL 是为了代表一张空的藏宝图
```

```
{
```

```
    printf("%3d",L->data);//打印当前结点数据
```

```
    L=L->next;//指向下一个结点
```

```
}
```

```
    printf("\n");
```



```
}
```

```
//《王道C 督学营》课程
```

```
int main()
```

```
{
```

```
LinkList L;//链表头, 是结构体指针类型
```

```
//list_head_insert(L);//输入数据可以为 3 4 5 6 7 9999,头插法新建链表
```

```
list_tail_insert(L);//输入数据可以为 3 4 5 6 7 9999
```

```
print_list(L);//链表打印
```

```
return 0;
```

```
}
```